Model performance

(i) classification — (i) Acc
(ii) CM
(iii) Pr, Recall, F1-score
(iv) log-loss (prob score)
(v) ROC (Receiver operating characteristics curve) and AUC (Area under

(ii) Regression

good/Bad

1 / 2

X	Actual Y	$\hat{Y}$	High $Y_{\sigma=0.95}$	$Y_{\sigma=0.92}$	$Y_{\sigma=0.80}$
X1	1	0.95	1	1	1
X2	1	0.92	0	1	1
X3	0	0.80	0	0	1
X4	1	0.76	0	0	0
X5	1	0.71	0	0	0

0.5 → 1
→ 0

∀ unique prob score

(sort-ac)

decreasing

(TPR, FPR)

(TPR, FPR)

(TPR, FPR)

if $\hat{Y} \geq 0.95$ → 1
else → 0

	0	1
0	1	3
1		

2 / 3

$\eta_1 \quad \eta_2 \quad \eta_3 \quad - - - - - - \quad \eta_n$

$\begin{pmatrix} TPR \\ FPR \end{pmatrix} \quad \begin{pmatrix} TPR \\ FPR \end{pmatrix} \quad \begin{pmatrix} TPR \\ FPR \end{pmatrix} \quad - - - \quad \begin{pmatrix} TPR \\ FPR \end{pmatrix}$

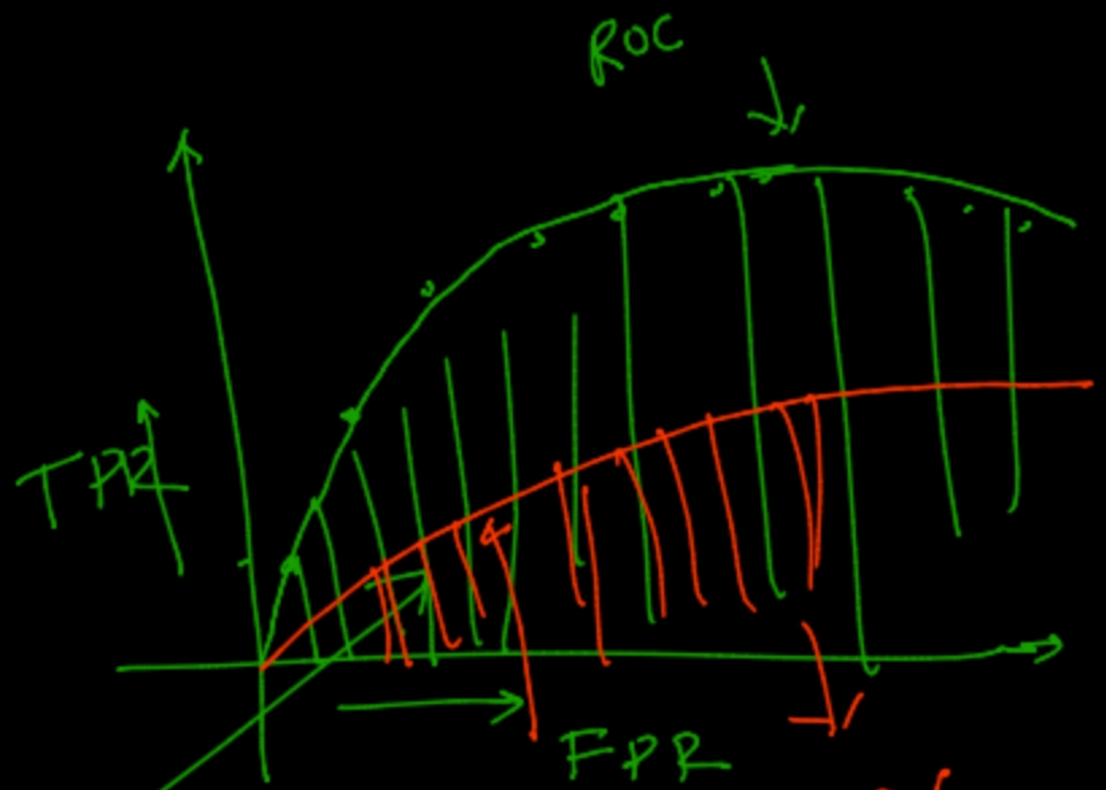
$\begin{pmatrix} 0.7 \\ 0.3 \end{pmatrix} \quad - - - -$

$\begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix}$

Sklearn → method → ROC & curve

sort → ≡

$\boxed{TPR\uparrow \quad FPR\uparrow}$



AUC →

0 — 0.5 — 1

terrible model

Best

50-1.

AUC

↓

0 — 1

↑ ↑

Best

0.5

sort.

↑

terrible model

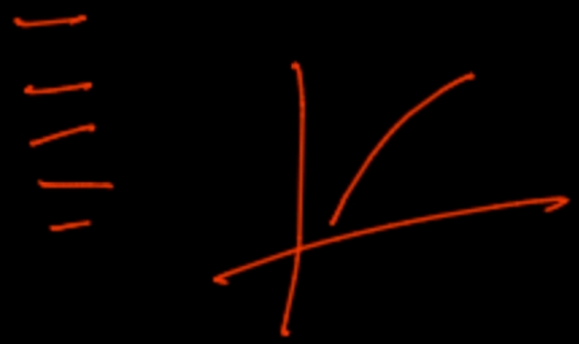
(i) Imbalance dataset

↓

X AUC →

(ii) AUC → Y

↳ ordering



4 / 4

classification

↓

(i) statement domain

data → define KPI

↓

model

↓

imbalanced

Pr, re, F1

↓

✓

project → cancer (minima)

(medical)

↑

RISK

↓

model confidence

↓

prob score

log-loss

project

minimal

0

0.4

40%

0.6 → 1

0.9 → loss

0.95 → ↓

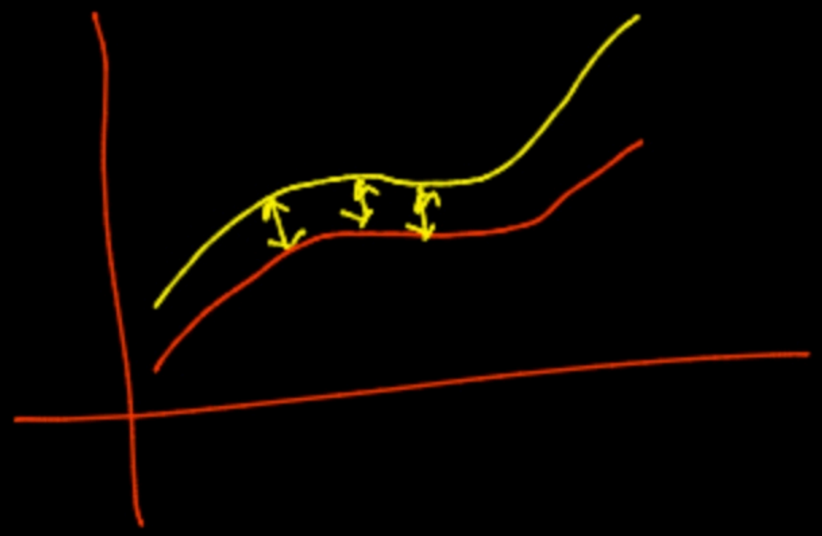
Kaggle

(in)

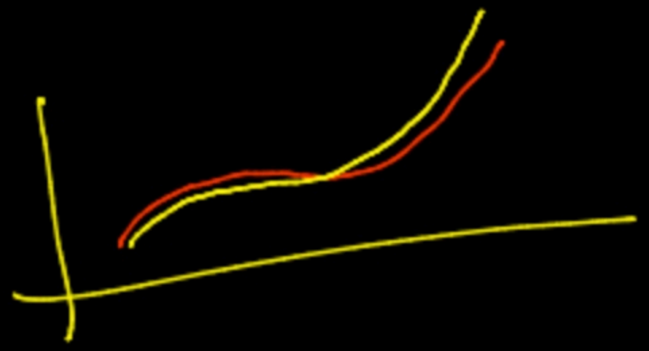
Regression

Y	$\hat{Y}$	error $(Y-\hat{Y})^2$
321	325	$(\ )^2$
451	100	$(\ )^2$
625	625	$(\ )$
125	—	$(\ )$
⋮	—	$(\ )$
⋮	—	$(\ )$
⋮	—	$(\ )$

M1 →



M2



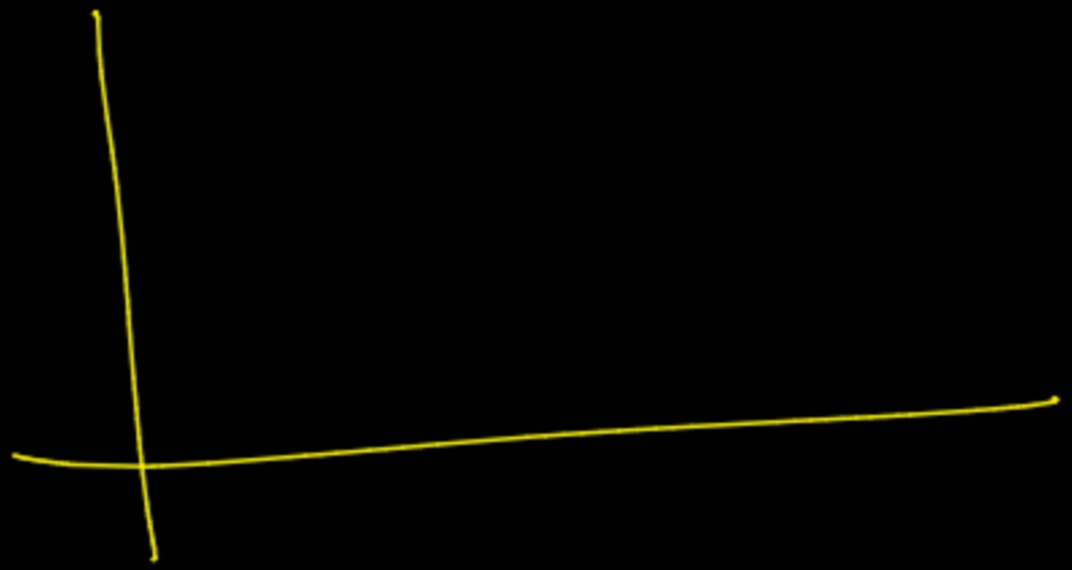
Mean

M1 → 3264 (error)

M2 → 2700

M3 → 1700

0 ||



case i

17	17	$\rightarrow (-2)^2 \Rightarrow \oplus$
16	16	$\Rightarrow 0$
15	15	$\Rightarrow 0$

$\oplus \uparrow$

(case ii)

	12	
17	22	(25) ✓
16	32	144
15	15	0

169 $\uparrow$

Linear Reg

$y_i \rightarrow 0$

Overfit

Und.

err

(i) MSE

(ii) MAE

(iii) MAPE (15%)

(iv) R^2 sq

(v) RMSE

→ (vi)

17 15 121

LR

9 / 9

KNN
Model KPI
↓
New Algorithm
↓
'Log Regression'
↓

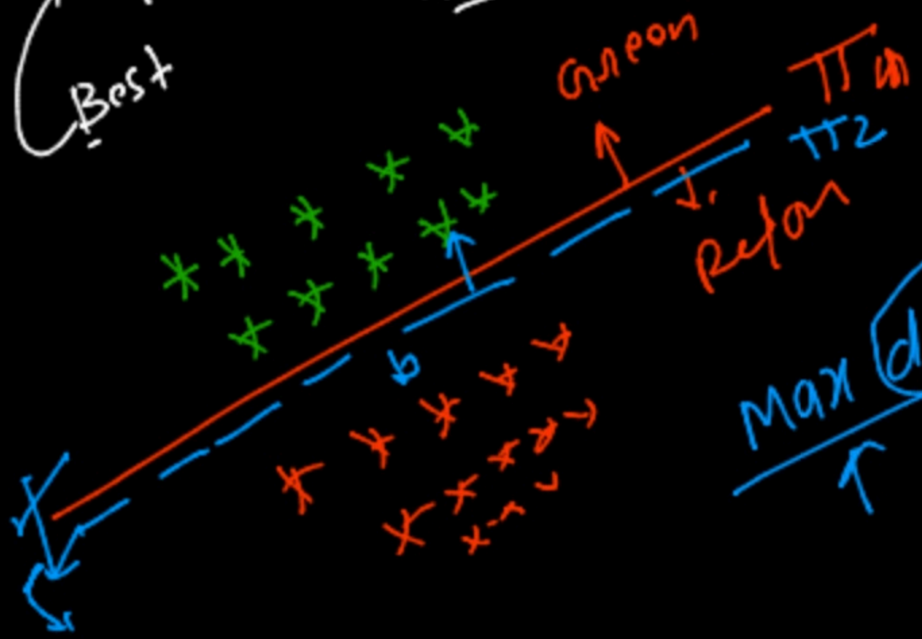
Logistic Regression → Assumption (Real (FE))

(i) classes are almost/perfectly (tangent) linearly separable

Geometry ← Best

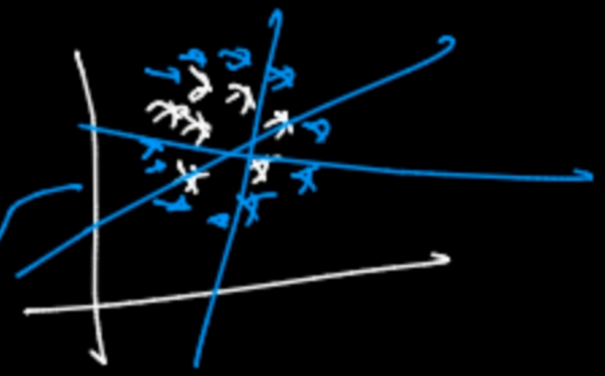
Prob ← MSC. stats

loss function

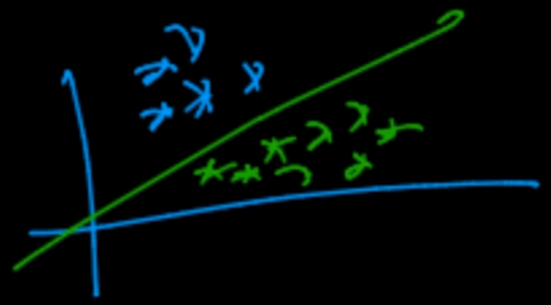


Max distance ← maximize

eg. $D_x(x, y)$ → plot it



FE =



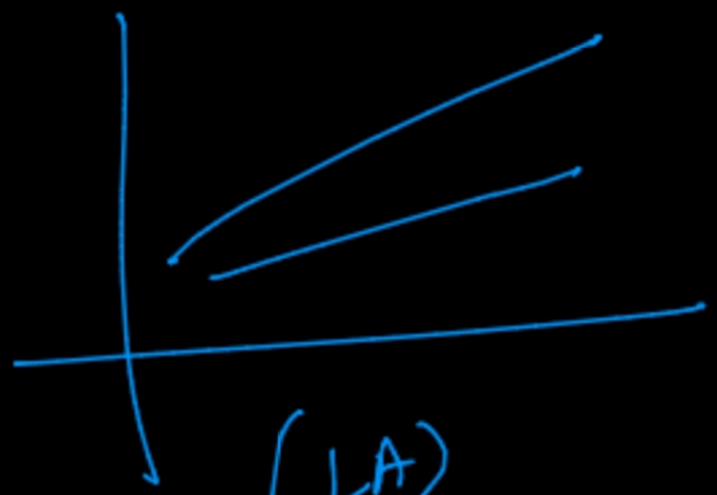
eq^n of plane

$$\pi : w^T \cdot \underline{\underline{x}} + b$$

$$D_n \Rightarrow \{+ve, -ve\}$$

$$\rightarrow [w, b]$$

Task



(LA)

Task : π that best separate +ve pts & -ve points

$\{w, b\}$ maximum distance

[0.1, 0.3

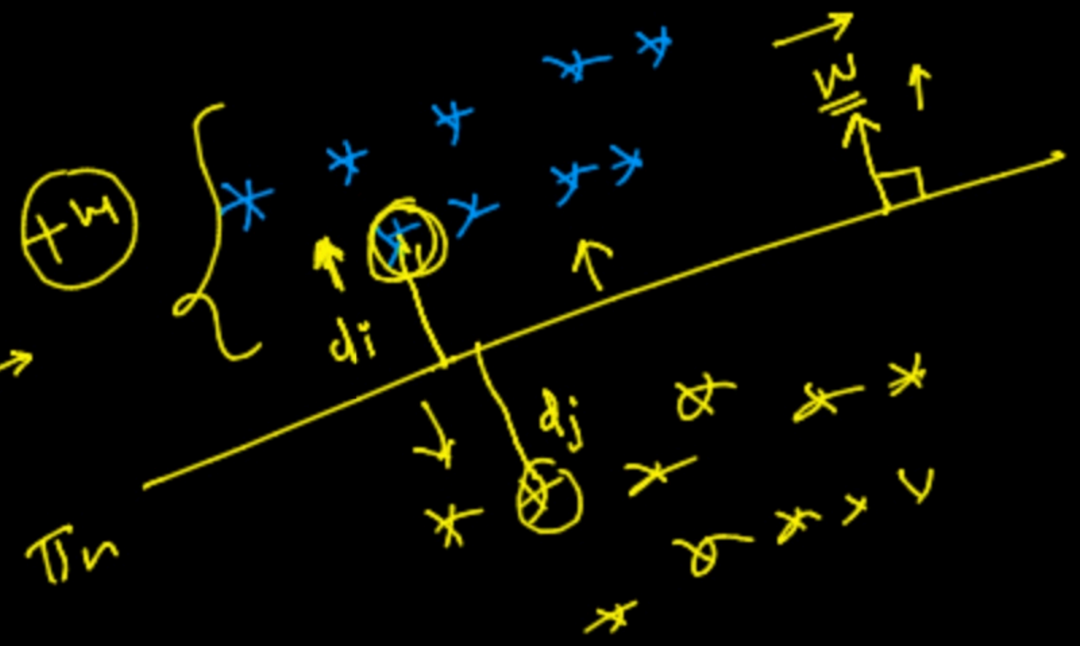
$$d_i = \frac{w^T \cdot x_i}{\|w\|}$$

(w is Normal to plane)

if $\|w\| = 1$

$$\boxed{d_i = w^T \cdot x_i}$$

$$d_j = w^T \cdot x_j$$



$w \rightarrow x_i$ (are same side)

$d_i = w^T \cdot x_i > 0$ (same side of Normal)

$d_j = w^T \cdot x_j < 0$ (opposite side of Normal)

$\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$

decision surface

LR

(i) classifier say

$$\rightarrow \begin{cases} \text{if } w^T \cdot x_i > 0 \\ \quad \text{then} \rightarrow y_+ \rightarrow +w \\ \text{if } w^T x_j < 0 \rightarrow -w \end{cases}$$

case i

$$Y_i * \underbrace{w^T \cdot x_i}_{\text{model}} > 0$$

(1) (1)

correct ☑

$\Downarrow$

for correct classification

$$\boxed{Y_i\, w^T x_i > 0}$$

$$Y_i \quad \underbrace{w^T \cdot x_i}$$

(1) (−1)

(−1)

$$Y_i = (-1)$$

$$\underbrace{(-1)\,(-1)}_{=+}$$

$y_i w^T \cdot x_i < 0 \rightarrow$ mis-classification

$\downarrow$

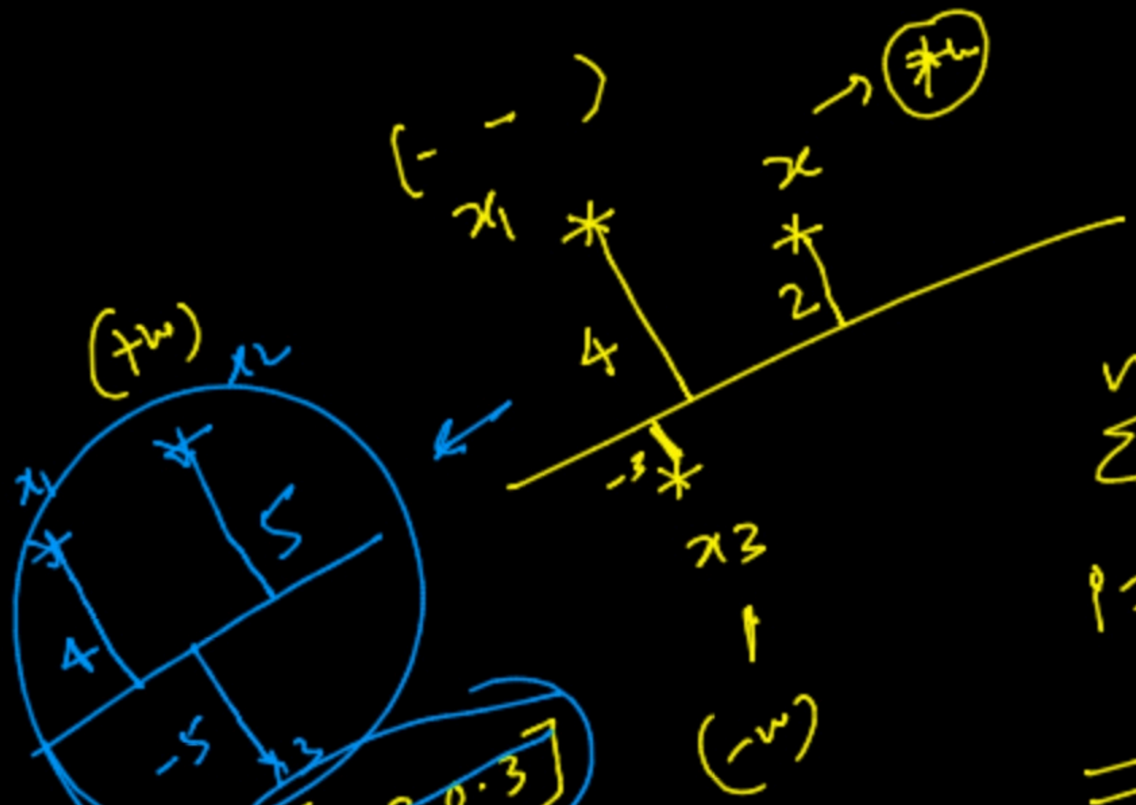
$w^* \xrightarrow{\text{which maximize}} \arg\max_{w} \left(\sum_{i=1}^{n} y_i (w^T \cdot x_i) \right)$

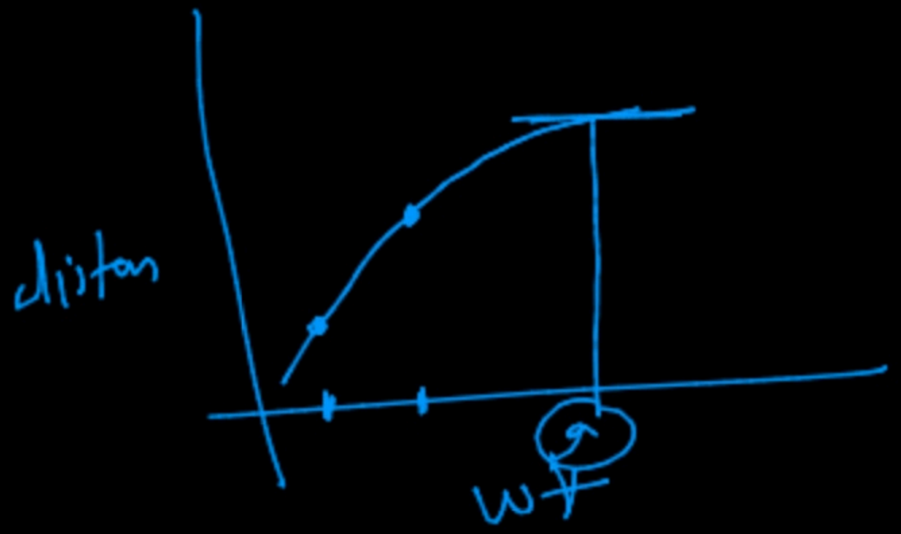
outliers

optimization problem

(stage 1)

λ_i





$$\sum_{i=1}^{n} y_i w^T x_i$$

$$= 4(1) + (2)(1)$$

$$4 + 2 + (-3)(-1)$$

$$\Rightarrow 9$$

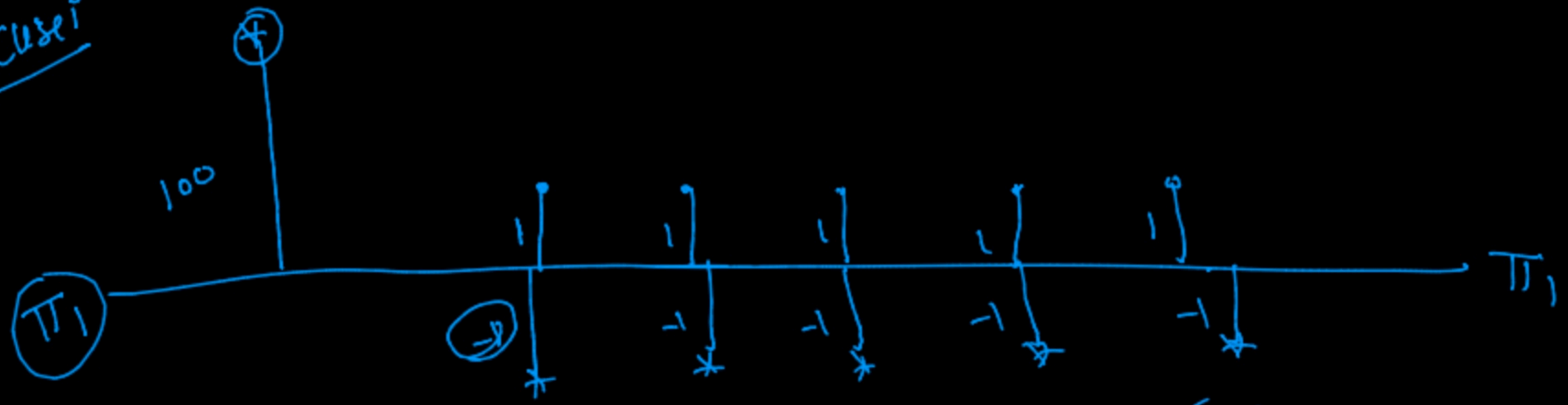
Algorithm

SGD

task

17 / 17

Case i



$$\frac{10}{11}$$

$$\sum_{i=1}^{n} y_i w^T x_i$$

$\Downarrow$

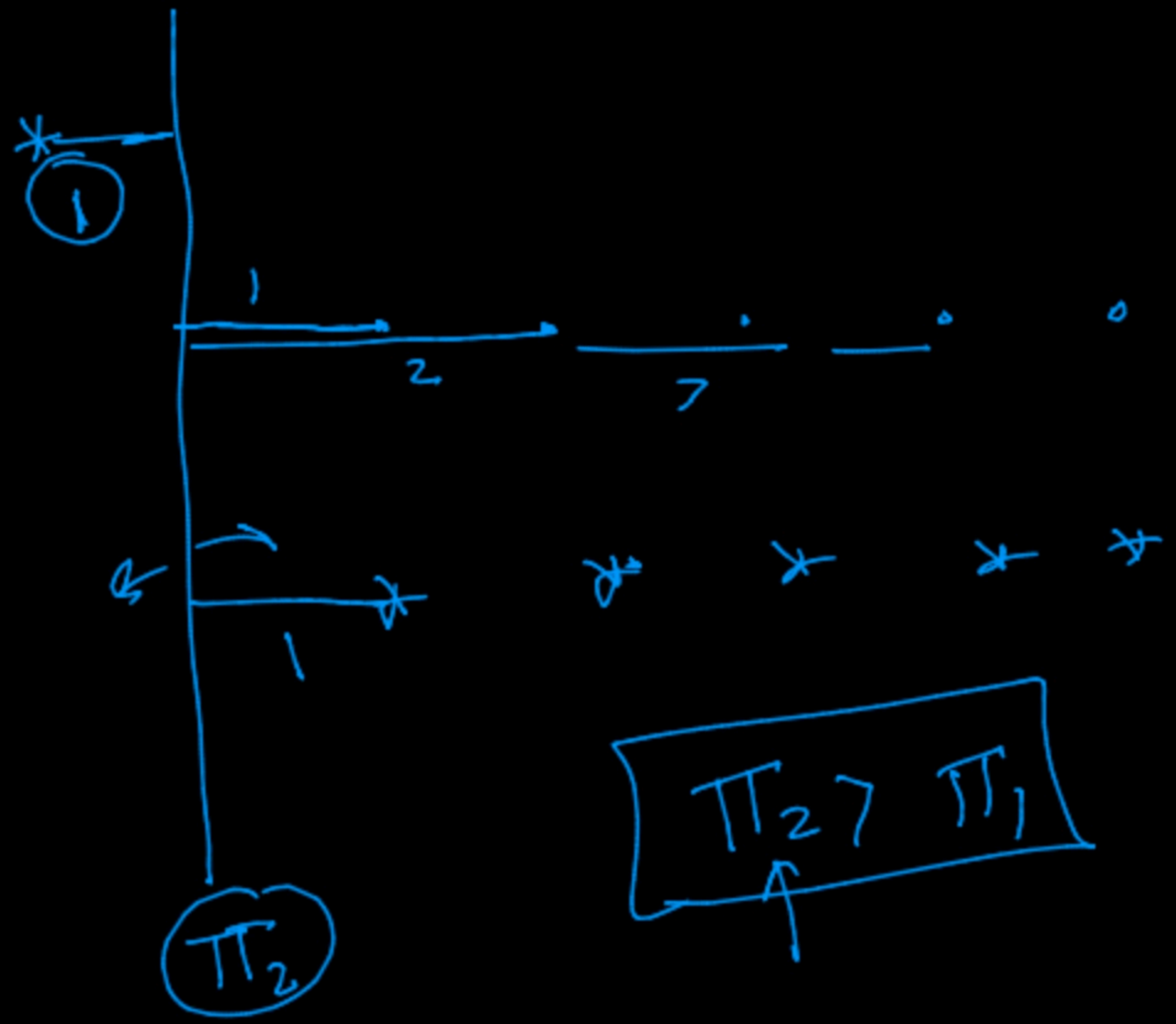
$1+1+1+1+1+1$

$5+5+100(-1)$

$10-100 \Rightarrow -90$

max weight

(case ii)



$\frac{6}{11}$

$y_i w^T x_i$

$\Downarrow$

$\frac{4+2+3+1+5+}{-5-2-3-4-5}$

$+1$

$\Downarrow$ 1

$\boxed{\pi_2 > \pi_1}$

Model

Sigmoid

$$f(x) = \frac{1}{1+e^{-x}}$$

80%

$$w^* = \underset{w}{\operatorname{argmax}} \sum_{i=1}^{n} \frac{1}{1+\exp(-y_i w^T x_i)}$$

10 code

+

(3A)

sklearn.linearmodel import LogReg

model = LogReg( )

model.fit(D)

...

fine

Grid